

IaC의 도구 Terraform에 들어가기 전에,

전통적인 배포 시나리오

- 비즈니스 쪽에서 새로운 제품에 대한 요구사항이 발생한다.
- 솔루션 아키텍트(architect)와 테크 리드(Tech Lead)가 제품을 중심으로 디자인을 하고 계획을 수립한다.
- 구매부서 승인을 받으면 장비를 구입하거나 리스를 한다.
- 주문 후 상당시간이 걸려서 장비가 들어오게 된다.
- 인프라쪽 일 진행
 - 온프레미스 IT 담당자, 엔지니어, 네트워크 담당자 등이 하드웨어를 설치
- Devops 환경 설정
 - 환경설정, 리눅스 설치, 애플리케이션 설치
- SW 엔지니어들이 Application을 배포

단점

☞ 비용과 사람도 많이 들어가고 6개월에서 1년까지 걸릴 수 있는 일이다.

요즘의 배포는,

가상화/클라우드 환경에서의 배포

- Devops 와 SW 엔지니어들만 있으면 효율적으로 배포가 가능하다.
- 동일한 HW의 숫자를 늘리는 스케일 아웃이 손쉽게 가능하다.

단점

- ☞ 여러 명의 엔지니어들이 클라우드 UI를 통해 이 화면 저 화면 에 있는 버튼을 눌러서 인프라를 제어
- ☞ 인프라 설정을 한 눈에 보거나 제어하기가 어렵다.
- ☞ 사용자의 Role 과 정책도 꼼꼼하게 잘 연결하지 않으면 모든 퍼미션을 가져서 문제 발생 가능
- ☞ 사용 중이 아닐 때는 수작업으로 삭제하거나 중지해야 한다. 그렇지 않으면 비용이 어디서 발생하는지도 확인하기가 어려워 질 수가 있다.

위의 가상화/클라우드 환경에서의 단점을 보완하기 위해서 IaC 도입되었다.

IaC (Infrastructure As Code)란?

“인프라를 코드로 정의하고 관리하는 방식”

즉, 서버, 네트워크, 보안, 데이터베이스 등을 사람이 클릭해서 설정하는 게 아니라,
코드로 작성하고 자동화하여 설정한다.

코드를 통해 관리하고 프로비저닝하는 것을 의미한다.

IaC는 코드를 활용하기 때문에 작성의 용이성, 재사용성, 유지보수의 용이성 등의 장점을 가진다.

왜 IaC를 사용할까? (필요성)

이유	설명
✅ 자동화	클릭 없이 코드 한 줄로 인프라 생성 가능 (DevOps 핵심)
✅ 버전 관리	Git으로 인프라 상태 추적 가능 → 롤백도 쉬움
✅ 재현 가능	동일한 인프라를 여러 환경(dev/test/prod)에 복사 생성 가능
✅ 사람 실수 최소화	수동 클릭보다 정확함 (오타, 누락 방지)
✅ 협업 효율성	개발자 간 코드로 인프라를 리뷰하고 공유 가능

IaC 도구의 종류

도구	특징
Terraform	가장 인기 있는 클라우드 플랫폼 독립 IaC 도구
AWS CloudFormation	AWS 전용 IaC 도구
Pulumi	다양한 언어(JS, Python 등)로 IaC 작성
Ansible	서버 설정 자동화에 특화 (상태 기반 아님)
CDK (AWS CDK)	실제 프로그래밍 언어로 인프라 구성 (TypeScript, Python 등)

Terraform이란?

“멀티 클라우드 인프라를 코드로 선언하고 자동으로 배포하는 도구”

Open Source Infrastructure As Code

HashiCorp사에서 만든 솔루션 중 하나이다.

Infrastructure를 생성하는 코드를 생성하면 이 코드가 퍼블릭 클라우드와 연결을 해서 Provider라는 객체를 통해 상호작용하면서 네트워크 설정이나 EC2 등을 생성해준다.

한번 작성해 놓은 코드를 이용해서 항상 동일하게 (역등성)인프라를 유지할 수가 있다.

AWS, Azure, GCP, Kubernetes 등 다양한 클라우드 인프라를 하나의 문법으로 제어 가능하다.

.tf 파일(HCL 언어)로 인프라를 정의하고 terraform apply로 자동 구축한다.

☞ [Terraform | HashiCorp Developer](https://developer.hashicorp.com/terraform) - <https://developer.hashicorp.com/terraform>

☞ [Terraform Product Brief_1P.ai](https://www.datocms-assets.com/2885/1618970795-terraform-product-briefdigitalko.pdf) - <https://www.datocms-assets.com/2885/1618970795-terraform-product-briefdigitalko.pdf>

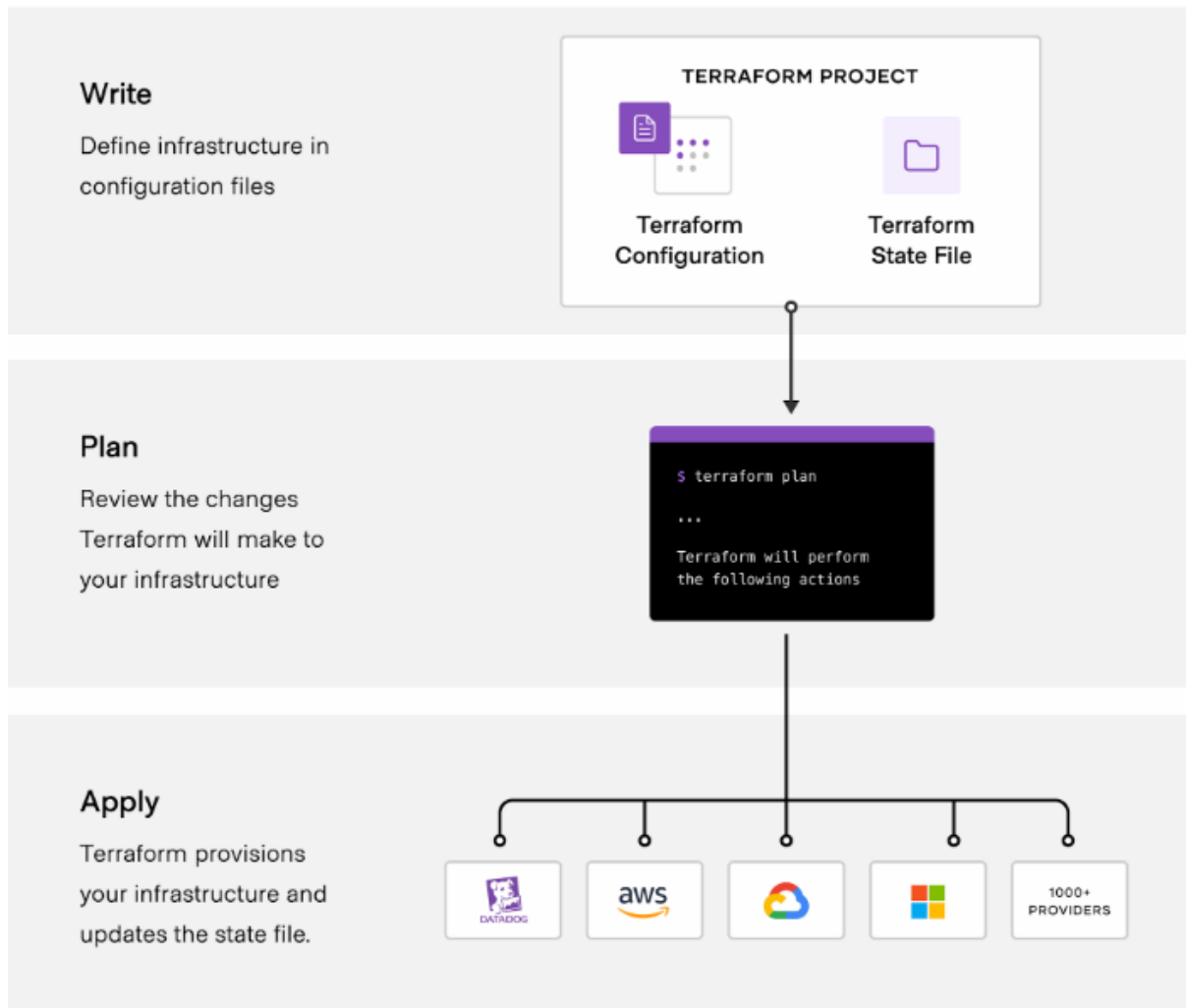
테라폼을 이용한 배포

- ✓ 개발자가 IAC 즉 인프라를 제어하는 Configuration File을 HCL 언어를 이용해 코드를 작성한다.
- ✓ S3, VPC 등을 별도로 모듈화를 해서 만든다.
- ✓ 테라폼이 실행 플랜을 빌드해서 보여준다. 즉 가상으로 실행을 해보는 것이라고 보면 된다.
- ✓ 에러가 없으면 Apply를 한다. 테라폼은 실행하면서 프로비저닝(필요한 만큼 자원을 자동 또는 수동으로 준비하고 사용할 수 있게 만드는 것)을 한다.

Terraform 기본 동작 구조

<https://developer.hashicorp.com/terraform/intro>

1. **Write** : 인프라 환경을 HCL언어로 리소스 정의
2. **Plan** : 정의된 리소스들을 생성, 수정, 삭제하기 위해 실행 계획을 확인
3. **Apply** : 확인된 실행 계획을 리소스 종속성을 고려하여 올바른 순서대로 수행



개발자가 code 를 작성 > IaC > plan 즉 ping 을 날려보고 문제가 없는지 결과값이 어떨지 plan 을 보고 output 을 샘플 코드를 본다.

Id를 제외하고는 실제와 동일한다.

- 문제 없으면 Apply를 한다.
- AWS나 구글 같은 프로바이더들이 기본적인 것을 제공을 한다. 프로바이더를 가지고 서비스를 만드는 것이다.

일관성 있고 신뢰성 있게 인프라를 관리할 수 있게 된다.

테라폼의 구성요소

provider	테라폼으로 생성할 인프라의 종류 - aws, gcp
resource	테라폼으로 실제 생성할 인프라 자원 - EC2 인스턴스, S3 버킷, VPC, RDS
state	테라폼을 통해 생성한 자원의 상태 - 테라폼코드로 만들어진 자원

	인프라의 현재 상태를 추적하고 관리 상태 정보를 기반으로 변경 사항을 계산하고 plan, apply, destroy 명령을 실행
output	테라폼으로 만든 자원을 변수형태로 state에 저장하는 것
module	공통적으로 활용할 수 있는 코드를 모듈 형태로 정의하는 것
remote	다른 경로의 state를 참조하는 것(output 변수를 불러올 때 사용)

테라폼으로 생성할 인프라의 종류 - aws, gcp

provider

provider.tf

```
// Configure the AWS Provider
provider "aws" {
  region = "ap-northeast-2"
  version = "~> 3.0"
}
```

provider 안에서 다양한 argument를 가짐
AWS resource를 다루기 위한 파일들을 다운로드 하는 역할을 함

테라폼으로 실제 생성할 인프라 자원

예시) 테라폼으로 VPC를 생성하는 코드

resource

main.tf, vpc.tf 등 원하는 형태의 파일이름 사용 가능

```
//aws vpc를 생성하는 역할의 resource
resource "aws_vpc" "sample_vpc" {
  cidr_block = var.vpc.cidr
  tags = {
    Name = var.vpc.Name
    terraform = var.vpc.terraform
  }
}

//main.tf에서 쓰이는 resource들에서 쓰이는 변수들을 모아놓은 맵 형식의 변수이다.
variable "vpc" {
  type = map(any)
  default = {
    cidr = "10.1.0.0/16"
    Name = "sample_vpc"
    terraform = "madeByTerraform jj"
  }
}
```

테라폼을 통해 생성한 자원의 상태 - 테라폼코드로 만들어진 자원

테라폼으로 작성된 파일을 실행하게 되면 생성되는 결과값을 가지고 있는 파일이다.

테라폼 state 는 현재 인프라의 상태를 의미하는 것은 아니다. 이 state 파일과 시스템 인프라의 상태를 똑같이 유지해야 한다.

state

terraform.tfstate 파일

```
{
  "version": 4,
  "terraform_version": "1.5.6",
  "serial": 1,
  "lineage": "3XXXXb4-eXX-d4e2-e2d7-41XXXXXXXX41",
  "outputs": {},
  "resources": [
    {
      [생략]
      "name": "sample-vpc",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "cidr_block": "10.1.0.0/16",
      "default_security_group_id": "sg-028b5400191f9b57f",
      "dhcp_options_id": "dopt-0b8e4f60",
      [생략]
    }
  ]
}
```

테라폼으로 만든 자원을 변수형태로 state에 저장하는 것

vpc를 테라폼파일로 만들게 되면 vpc id 나 cidr 값이 생기는데 변수를 state 파일로 저장을 한다. remote를 사용해서 재사용을 할 수 있다.

output # outputs.tf 파일

```

output "vpc_id" {
  value = {
    VPC_ID              = aws_vpc.sample_vpc.id,
    default_RouteTable_id = aws_vpc.sample_vpc.default_route_table_id
  }
}

output "subnet_id" {
  value = [
    aws_subnet.test_subnet
  ]
}

output "gateway_id" {
  value = aws_internet_gateway.gw.id
}

```

15

공통적으로 활용할 수 있는 코드를 모듈 형태로 정의하는 것

- NAME : 테라폼 코드 전체에서 참조하기 위해 사용하는 식별자
- SOURCE : 모듈 코드를 찾을 수 있는 경로
- CONFIG : 모듈과 관련된 특정한 하나 이상의 인수로 구성

모듈을 어떻게 쓰느냐에 따라 테라폼을 굉장히 잘 쓰는지를 판단할 수 있다.

module # terraform.tfstate 파일

```

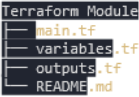
module "<NAME>" {
  source = "<SOURCE>"

  [CONFIG ...]
}

module "vpc" {
  source = "../_modules/vpc"

  cidr_block = "10.1.0.0/16"
}

```

 Terraform Module directory structure: main.tf, variables.tf, outputs.tf, README.md

module은 한 번 만들어진 코드를 같은 형태로 반복적으로 만들어낼 때 주로 사용

다른 경로의 state를 참조하는 것(output 변수를 불러올 때 사용)

```
remote      #
            data "terraform_remote_state" "vpc" {
              backend = "remote"

              config = {
                bucket = "terraform-s3-bucket"
                region = "ap-northeast-2"
                key    = "terraform/vpc/terraform.tfstate"
              }
            }

remote state는 원격 참조인데 config 정보에 저장된 값을 참조할 수 있음을 의미
함, 명시된 state 에서 변수를 가져오게 된다.
```

Terraform 기본 명령어

init	테라폼 명령어 사용을 위한 각종 설정 진행
plan	테라폼으로 작성한 코드의 실행시 예측 결과를 보여줌
apply	테라폼 코드로 실제 인프라를 생성하는 명령어
import	이미 만들어진 자원을 테라폼 state 파일로 옮기는 명령어
state	테라폼 state를 다루는 명령어, mv, push 와 같은 명령어가 있음
destroy	생성된 자원들을 state 파일 기준으로 모두 삭제하는 명령어



1. 앞단에 작성한 코드가 있고, init 명령어를 먼저 실행한다.
ex) provider, state, module 에 대한 설정
2. 예측 결과를 미리 확인하고 문제가 없다면(plan)
3. apply를 한다. 실제 인프라에 영향을 미치므로 반드시 plan 으로 확인하는 습관을 들여야 한다.

plan이 100% 보장해주지는 않는다. 인프라를 직접 생성한다는 것은 큰 결과와 비용으로 돌아올 수 있는 일이기 때문에 신중해야 한다.

이밖 에도 많은 명령어가 있지만 기본명령어가 90%이상 많이 사용한다.

테라폼의 유용한 명령어들

validate : 작성 중 문법 오류 확인(문법 오류가 있을 때 라인번호와 어느 부분이 잘못 됐는지 자세하게 보여줌, 오류가 없으면 success 라고 나온다)

- terraform validate

fmt : pretty format 적용, 가독성을 높여줌(명령어를 사용하면 모든 것이 다 정렬이 잘 되어 있는 경우에는 아무것도 결과는 안 나오지만(void) 정렬이 진행된 소스파일이 있는 경우에는 해당 소스파일이 보여지고 가서 보면 정렬이 잘 되어 있는 것을 볼 수 있다.)

- terraform fmt

show : 메타 정보 조회

- terraform show

providers : 프로바이더의 버전과 인포메이션 조회

- terraform providers

graph : graph 를 통해 종속성 및 생성할 리소스에 대한 관계를 시각적으로 조회

- DOT라는 그래프 설명 언어가 출력됨
- terraform graph

Terraform의 특성

- **Mutable Resource**(변동 리소스)

기존의 리소스를 destroy 하지 않고 속성을 변경해서 인프라 상태를 변경하는 방식

예) EC2 인스턴스를 만든 후 없애지 않고도 Security Group이나 Tag를 변경할 수 있음

장점 : 기존 리소스를 직접 수정하므로 변경 과정이 빠름

단점 : 변경 과정을 추적하기 어려워 이전 상태로 복구하는 것이 어려움

- **Immutable Resource**(불변 리소스)

기존 리소스를 변경할 수 없기 때문에 destroy 한 후에 새로 생성해야 함

예> S3 의 버킷명을 변경 할 때 기존 S3 버킷 자체가 없어지고

새로 생성된 버킷으로 대체됨

장점 : 이전 버전의 리소스를 쉽게 복구할 수 있음

단점 : 스토리지를 재생성하면 데이터가 유실될 수 있음(내부 데이터 백업.복원 필수)

- 변경에 민감한 리소스는 분리해서 배포하거나 수동 관리를 고려하는 것이 좋음

Terraform은 내부적으로 각 리소스의 속성이 "ForceNew = true"인지 확인

- `ForceNew = false` → **Mutable Update**
- `ForceNew = true` → **Immutable Replace (destroy + create)**

실습환경 세팅

1. AWS CLI 설치 및 설정
2. Terraform 설치 및 설정

AWS CLI(Command Line Interface)란?

AWS 서비스를 명령어로 제어할 수 있는 도구

- AWS 리소스를 콘솔 없이 명령어로 생성/조회/수정/삭제
- 서버 설정, S3 업로드, EC2 인스턴스 실행 등 모두 가능
- 윈도우, 맥, 리눅스 모두 지원

예시)

작업 예시	명령어
EC2 인스턴스 목록 보기	<code>aws ec2 describe-instances</code>
S3에 파일 업로드	<code>aws s3 cp myfile.txt s3://my-bucket/</code>
Lambda 함수 배포	<code>aws lambda update-function-code ...</code>
IAM 사용자 생성	<code>aws iam create-user --user-name testuser</code>
CloudWatch 로그 확인	<code>aws logs get-log-events ...</code>

Terraform을 하기 위해서 AWS CLI설치하는 이유?

Terraform은 자체적으로 AWS API를 호출할 수 있지만, 인증 정보(AWS 자격 증명)를 설정해야 하는데 이때 `aws configure` 명령을 제공하는 AWS CLI가 있으면 훨씬 간편하고 실수도 줄일 수 있다.

AWS CLI 설치하기

참고 : https://docs.aws.amazon.com/ko_kr/cli/latest/userguide/getting-started-install.html

AWS CLI 설치 및 업데이트 지침

설치 지침은 해당 운영 체제에 대한 섹션을 참조하세요.

▶ Linux

▶ macOS

▶ Windows

AWS CLI 설치 또는 업데이트

Windows에서 AWS CLI의 현재 설치를 업데이트하려면 업데이트할 때마다 새 설치 관리자를 다운로드하여 이전 버전을 덮어씁니다. AWS CLI는 정기적으로 업데이트됩니다. 최신 버전이 출시된 시기를 확인하려면 [GitHub에서 AWS CLI 버전 2 변경 로그](#)를 참조하세요.

1. Windows용 AWS CLI MSI 설치 관리자(64비트)를 다운로드하여 실행합니다.

<https://awscli.amazonaws.com/AWSCLIV2.msi>

또는 `msiexec` 명령을 실행하여 MSI 설치 관리자를 실행할 수 있습니다.

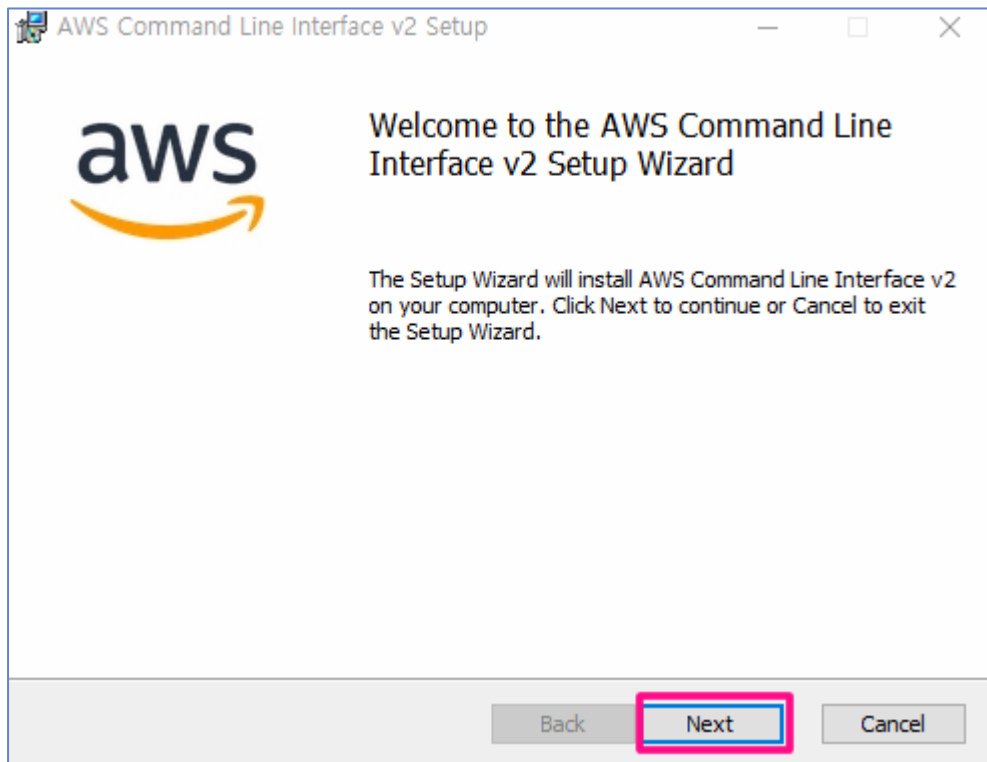
```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```

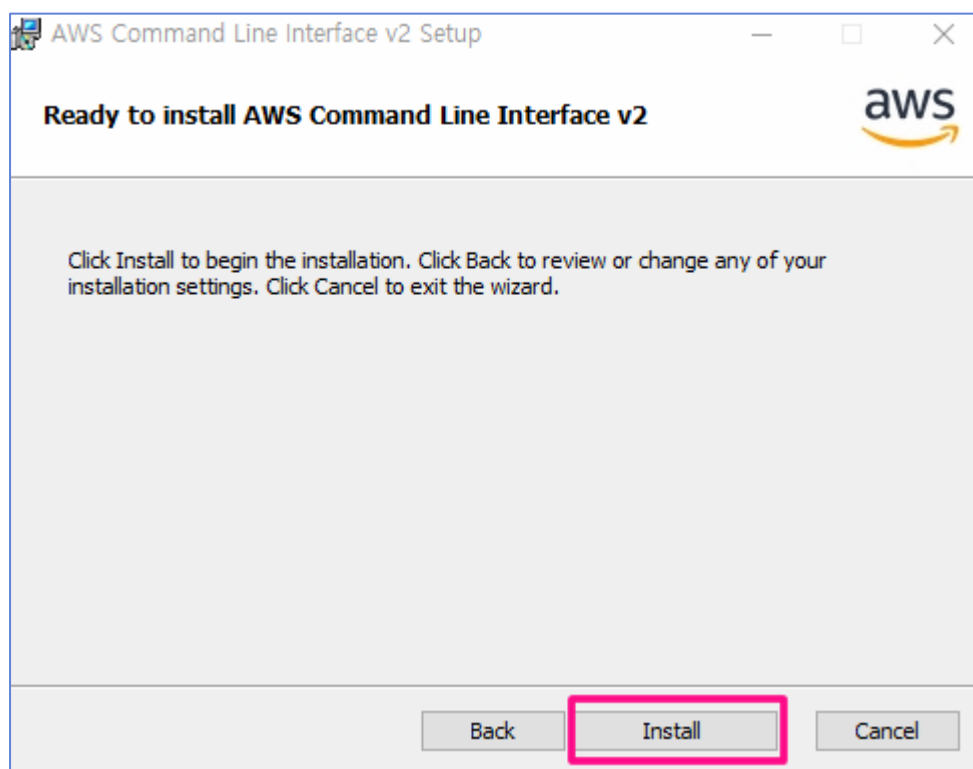
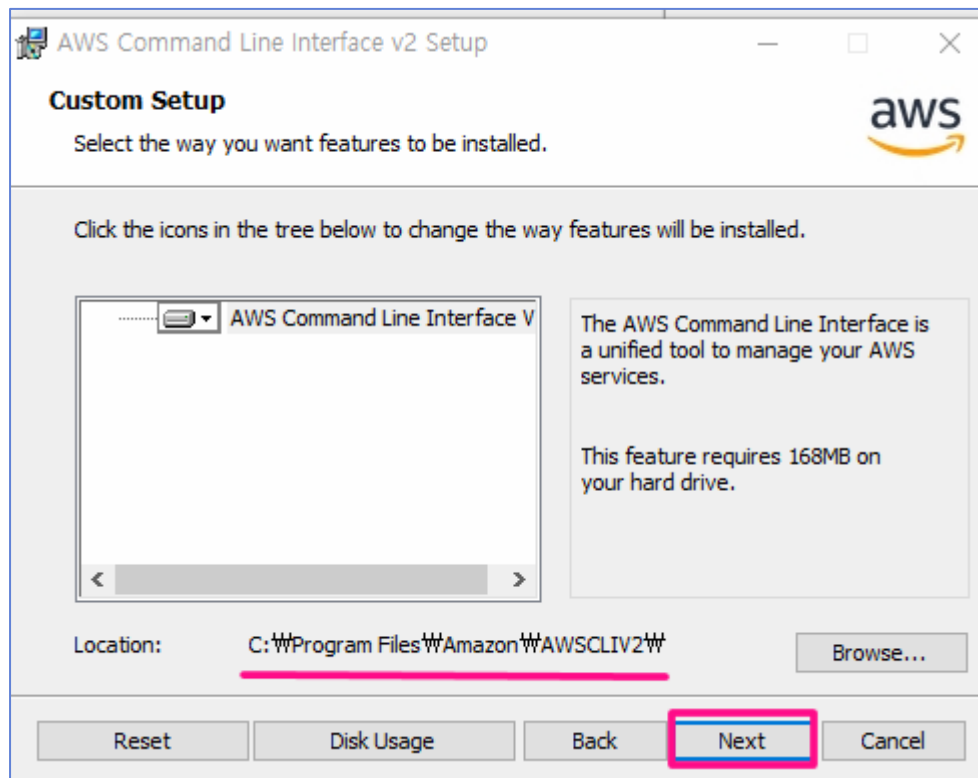
`msiexec`에서 사용할 수 있는 다양한 파라미터는 *Microsoft Docs* 웹 사이트에서 [msiexec](#)을 참조하세요. 예를 들어 자동 설치 시 `/qn` 플래그를 사용할 수 있습니다.

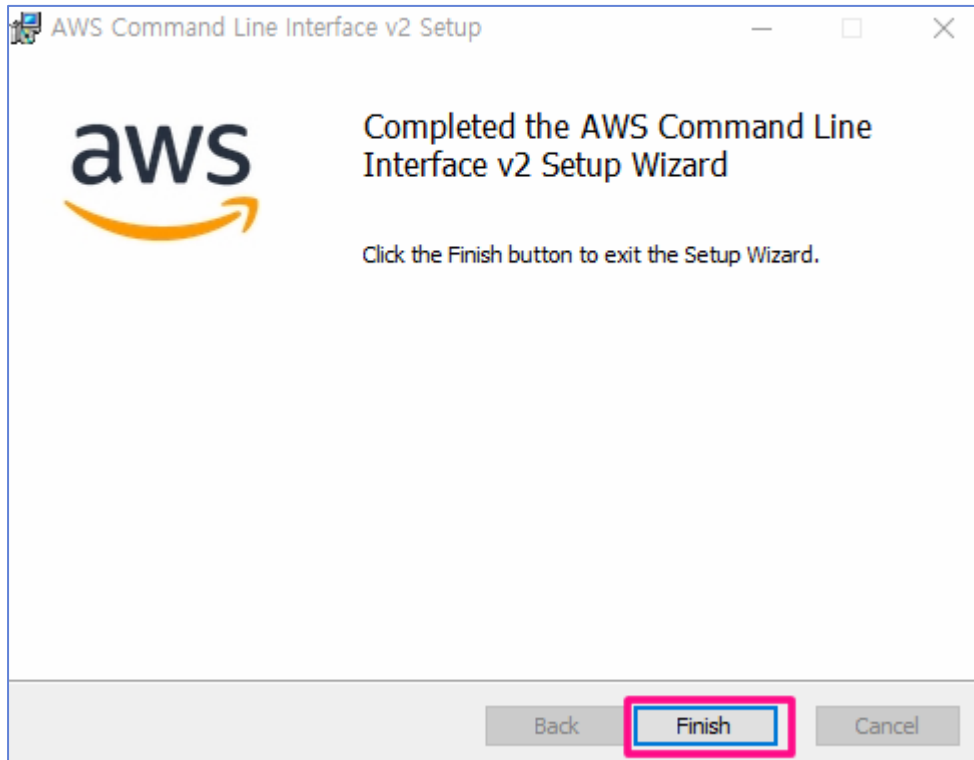
```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi /qn
```

Powershell을 관리자 권한으로 열고 위 참고 사이트에서 복사해서 설치한다.

```
msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```







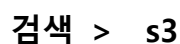
설치 완료후, 새로운 cmd창을 열고 version을 확인해본다.

```
aws --version
```

```
C:\Users\WKOSTA>aws --version  
aws-cli/2.27.49 Python/3.13.4 Windows/10 exe/AMD64
```

사용자 생성

1. 루트 계정으로 로그인하기
2. IAM 메뉴로 이동 후 사용자 생성하기
3. 사용자 이름 입력하고 management console로 로그인 할 수 있도록 체크박스 체크/IAM 사용자 생성/자동생성된 암호 사용하려면 선택하고 다음 클릭



IAM > 사용자 > 사용자 생성

- 1단계 사용자 세부 정보 지정
- 2단계 **권한 설정**
- 3단계 검토 및 생성
- 4단계 암호 검색

권한 설정

기존 그룹에 사용자를 추가하거나 새 그룹을 생성합니다. 직무별로 사용자의 권한을 관리하려면 그룹을 사용하는 것이 좋습니다. 자세히 알아보기

권한 옵션

☐ 그룹에 사용자 추가

기존 그룹에 사용자를 추가하거나 새 그룹을 생성합니다. 그룹을 사용하여 직무별로 사용자 권한을 관리하는 것이 좋습니다.

☐ 권한 복사

기존 사용자의 모든 그룹 멤버십, 연결된 관리형 정책 및 인라인 정책을 복사합니다.

☒ 직접 정책 연결

관리형 정책을 사용자에게 직접 연결합니다. 사용자에게 연결하는 대신, 정책을 그룹에 연결한 후 사용자를 직접한 그룹에 추가하는 것이 좋습니다.

권한 정책 (2/1369)

새 사용자에게 연결할 정책을 하나 이상 선택합니다.

정책 이름	유형	연결된 엔터티
☐ AmazonDMSRedshiftS3Role	AWS 관리형	0
☒ AmazonS3FullAccess	AWS 관리형	0
☐ AmazonS3ObjectLambdaExecutionRolePolicy	AWS 관리형	0
☐ AmazonS3OutpostsFullAccess	AWS 관리형	0

VPC IAM S3 EC2 Aurora and RDS Route 53 Certificate Manager 정책 및 비유 관리 CloudFront CodeDeploy

IAM > 사용자 > 사용자 생성

- 1단계 사용자 세부 정보 지정
- 2단계 권한 설정
- 3단계 **검토 및 생성**
- 4단계 암호 검색

검토 및 생성

선택 사항을 검토합니다. 사용자를 생성한 후 자동 생성된 암호를 보고 다운로드할 수 있습니다(활성화된 경우).

사용자 세부 정보

사용자 이름

kosta-test

콘솔 암호 유형

Autogenerated

암호 재설정 필요

예

권한 요약

이름	유형	다음과 같이 사용
AdministratorAccess	AWS 관리형 - 직무	권한 정책
AmazonS3FullAccess	AWS 관리형	권한 정책
IAMUserChangePassword	AWS 관리형	권한 정책

태그 - 선택 사항

태그는 리소스를 식별, 구성 또는 검색하는 데 도움이 되도록 AWS 리소스에 추가할 수 있는 키 값 페어입니다. 이 사용자와 연결된 태그를 선택합니다.

리소스와 연결된 태그가 없습니다.

새 태그 추가

최대 50개의 태그를 더 추가할 수 있습니다.

취소

이전

사용자 생성

IAM > 사용자 > 사용자 생성

☒ 사용자가 성공적으로 생성됨

AWS Management Console에 로그인하기 위한 사용자의 암호와 이메일 지참을 보고 다운로드할 수 있습니다.

사용자 보기

- 1단계 사용자 세부 정보 지정
- 2단계 권한 설정
- 3단계 검토 및 생성
- 4단계 **암호 검색**

암호 검색

아래에서 사용자의 암호를 보고 다운로드하거나 AWS 관리 콘솔에 로그인하기 위한 사용자 지참을 이메일로 보낼 수 있습니다. 지금은 이 암호를 확인 및 다운로드할 수 있는 유일한 시간입니다.

콘솔 로그인 세부 정보

콘솔 로그인 URL

<https://288761761976.signin.aws.amazon.com/console>

사용자 이름

kosta-test

콘솔 암호

8L-vH4)e 숨기기

이메일 로그인 지참

취소

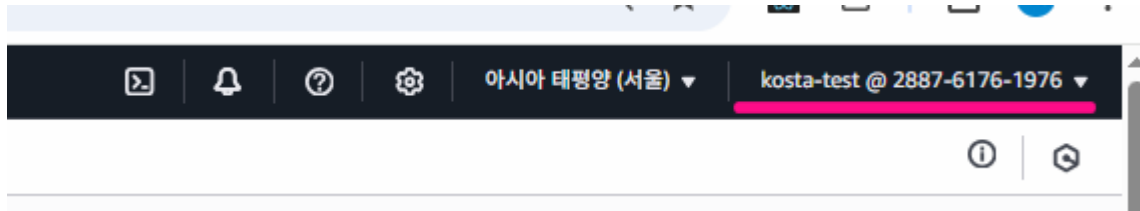
.csv 파일 다운로드

사용자 목록으로 돌아가기

.csv를 다운로드하면
아이디와 비밀번호가 들어있다

접속하면

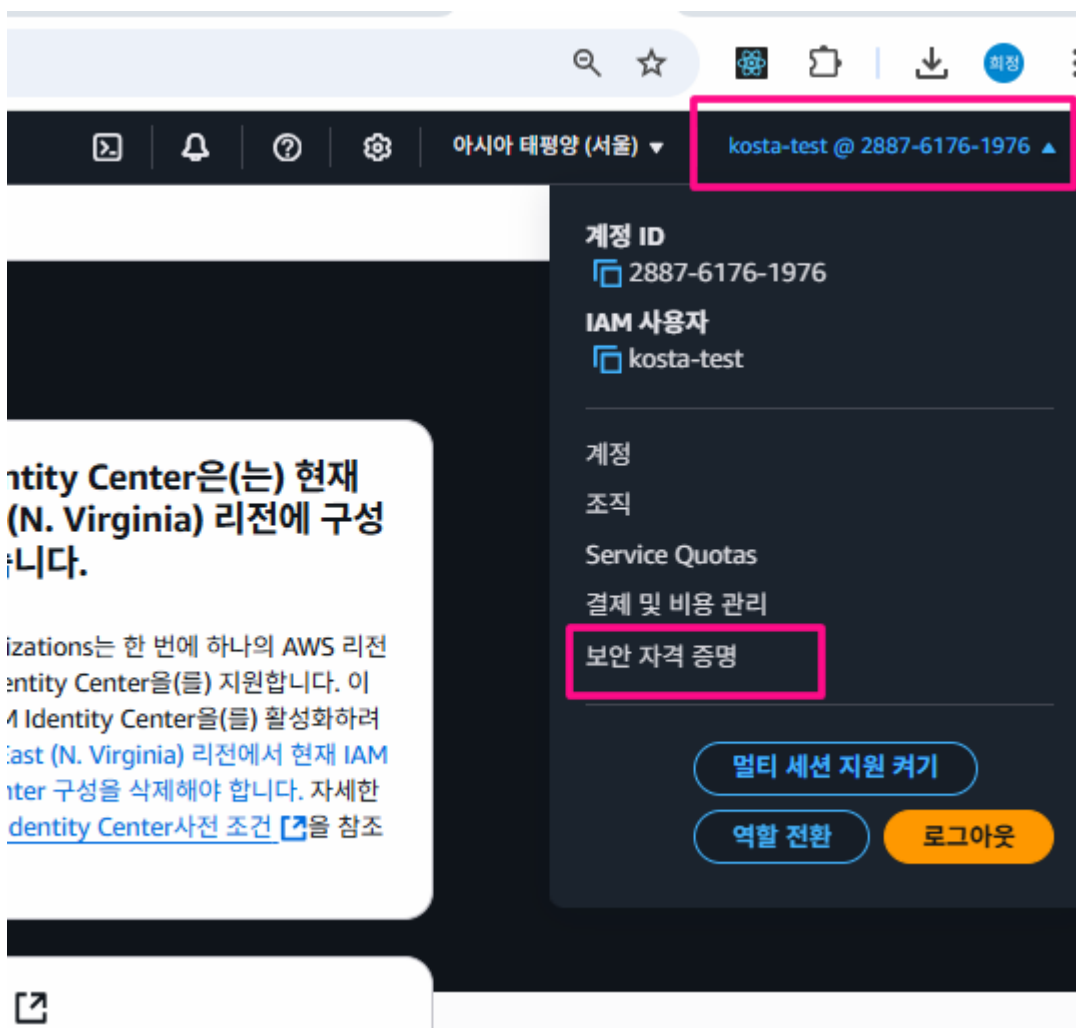
메모장에
계정ID 12자리
사용자이름
비밀번호
저장해 놓는다.



엑세스키 생성하기

Aws를 접근할 수 있는 권한이 필요하므로 우리의 iam 계정에 보안자격증명으로 권한을 부여받아야 한다.

계정 클릭하고 보안자격증명 선택한 후 엑세스키 생성하고
csv 다운로드 받아놓기



IAM > 보안 자격 증명

Identity and Access Management(IAM)

IAM 콘솔

시도도

IAM 관리

용자 그룹

용자

할

책

계급업제

정 설정

트 액세스 관리 [링크](#)

고서 액세스

Access Analyzer

리소스 분석 [링크](#)

미사용 액세스

분석기 설정

계급명 보고서

지 활동

비즈 제어 정책

소스 제어 정책 [링크](#)

M 자격 증명 센터 [링크](#)

OS Organizations [링크](#)

개입 애플리케이션

사용자 이름

kosta-test

AWS 계정 ID

288761761976

사용자 ARN

arn:aws:iam::288761761976:user/kosta-test

정규 사용자 ID

749258024f636f6710ddeda7a505aafdc376335453537c25d9fe5cc3cb7ff791

AWS IAM 자격 증명 | AWS CodeCommit 자격 증명 | Amazon Keyspaces 자격 증명 | Amazon Bedrock API 키

콘솔 로그인

콘솔 로그인 링크

<https://288761761976.signin.aws.amazon.com/console>

콘솔 암호

임대여도 10분 전 (2025-07-07 16:13 GMT+9)

마지막 콘솔 로그인

15분 전 (2025-07-07 16:08 GMT+9)

[콘솔 암호 업데이트](#)

멀티 팩터 인증(MFA) (0)

MFA를 사용하여 AWS 콘솔에 접근할 수 있습니다. MFA로 로그인하려면 MFA 디바이스의 인증 코드가 필요합니다. 각 사용자는 MFA 디바이스를 최대 5개까지 할당할 수 있습니다. [자세히 알아보기](#)

설정 | 재동기화 | **MFA 디바이스 할당**

용량 | 식별자 | 인증 | 생성 날짜

MFA 디바이스가 없습니다. MFA 디바이스를 할당하여 AWS 환경 보안 개선하기

[MFA 디바이스 할당](#)

엑세스 키 (0)

엑세스 키를 사용하여 AWS CLI, AWS Tools for PowerShell, AWS SDK 또는 직접 AWS API 호출을 통해 AWS에 프로그래밍 방식 호출을 전송합니다. 한 번에 최대 두 개의 액세스 키(활성 또는 비활성)를 가질 수 있습니다. [자세히 알아보기](#)

엑세스 키가 없습니다. 액세스 키와 같은 장기 보안 인증을 사용하지 않는 것이 좋습니다. 대신 단기 보안 인증을 제공하는 도구를 사용하세요. [자세히 알아보기](#)

[엑세스 키 만들기](#)

IAM > 보안 자격 증명 > 액세스 키 만들기

1단계

엑세스 키 모범 사례 및 대안

2단계 - 선택 사항

설명 태그 설정

3단계

엑세스 키 검토

엑세스 키 모범 사례 및 대안 [정보](#)

보안 개선을 위해 액세스 키와 같은 장기 자격 증명을 사용하지 마세요. 다음과 같은 사용 사례와 대안을 고려하세요.

사용 사례

☒ **Command Line Interface(CLI)**
AWS CLI를 사용하여 AWS 계정에 액세스할 수 있도록 이 액세스 키를 사용할 것입니다.

☐ 로컬 코드
로컬 개발 환경의 애플리케이션 코드를 사용하여 AWS 계정에 액세스할 수 있도록 이 액세스 키를 사용할 것입니다.

☐ AWS 컴퓨팅 서비스에서 실행되는 애플리케이션
Amazon EC2, Amazon ECS 또는 AWS Lambda와 같은 AWS 컴퓨팅 서비스에서 실행되는 애플리케이션 코드를 사용하여 AWS 계정에 액세스할 수 있도록 이 액세스 키를 사용할 것입니다.

☐ 서버 파티 서비스
AWS 리소스를 모니터링 또는 관리하는 서버 파티 애플리케이션 또는 서비스에 액세스할 수 있도록 이 액세스 키를 사용할 것입니다.

☐ AWS 외부에서 실행되는 애플리케이션
이 액세스 키를 사용하여 AWS 리소스에 액세스해야 하는 AWS 외부의 데이터 센터 또는 기타 인프라에서 실행 중인 워크로드를 인증할 것입니다.

☐ 기타
귀하의 사용 사례가 여기에 나열되어 있지 않습니다.

권장되는 대안

- 브라우저 기반 CLI인 [AWS CloudShell](#)을 사용하여 명령을 실행합니다. [자세히 알아보기](#)
- [AWS CLI V2](#)를 사용하고 IAM 자격 증명 센터의 사용자를 통한 인증을 활성화합니다. [자세히 알아보기](#)

확인

☒ 위의 권장 사항을 이해했으며 액세스 키 생성을 계속하려고 합니다.

[취소](#) [다음](#)

IAM > 보안 자격 증명 > 액세스 키 만들기

1단계

엑세스 키 모범 사례 및 대안

2단계 - 선택 사항

설명 태그 설정

3단계

엑세스 키 검토

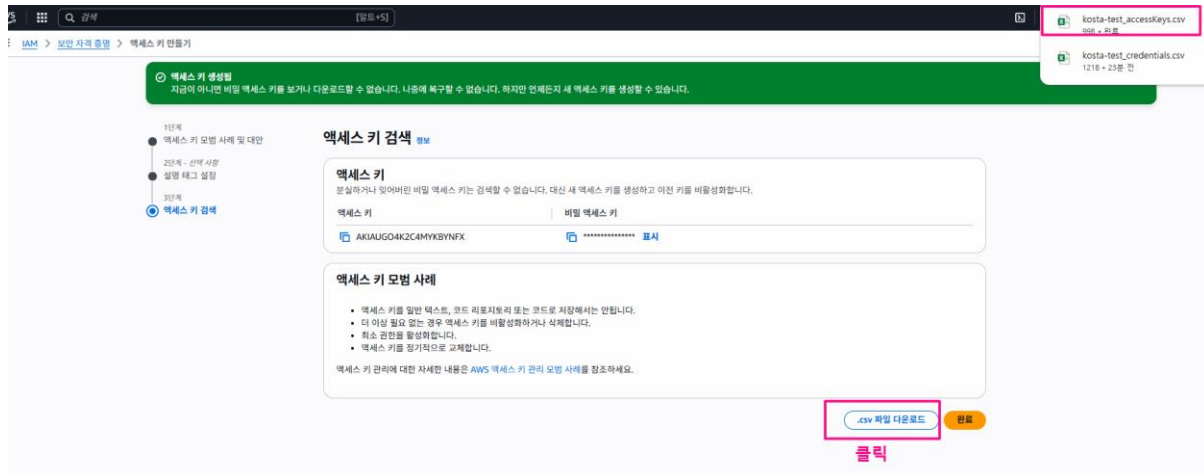
설명 태그 설정 - 선택 사항 [정보](#)

이 액세스 키에 대한 설명은 이 사용자에게 태그로 연결되고, 액세스 키와 함께 표시됩니다.

설명 태그 값
이 액세스 키의 용도와 사용 위치를 설명합니다. 좋은 설명은 나중에 이 액세스 키를 자신있게 교체하는 데 유용합니다.

최대 256자까지 가능합니다. 허용되는 문자는 문자, 숫자, UTF-8로 표현할 수 있는 공백 및 _ : / * * @입니다.

[취소](#) [이전](#) [엑세스 키 만들기](#)



aws configure라는 명령으로 액세스키와 시크릿키를 설정해주고 region을 서울 리전으로 지정한다.

포맷은 텍스트나 json을 사용하는데 우리는 json을 선택한다. 액세스키와 시크릿키는 절대 공개해서도 안되고 분실해서도 안된다. 만약 분실하거나 어딘가에 공개된 거 같다 라고 하면 액세스키 삭제하고 다시 생성해서 (그 사용자의 보안자격증명을 새로 받아서) 사용하면 된다.

```
C:\Users\WKOSTA>aws configure
AWS Access Key ID [None]: AKIAUGO4K2C4MYKBYNFX
AWS Secret Access Key [None]: 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
Default region name [None]: ap-northeast-2
Default output format [None]: json
```

명령어 > aws configure

다운받는 .csv 파일을 열어서 Access Key, Secret Access Key 설정

Region name : ap-northeast-2

Output format : json

이제, CLI설정이 잘 되었는지 확인하기 위해 CLI에서 bucket을 생성 해보자.

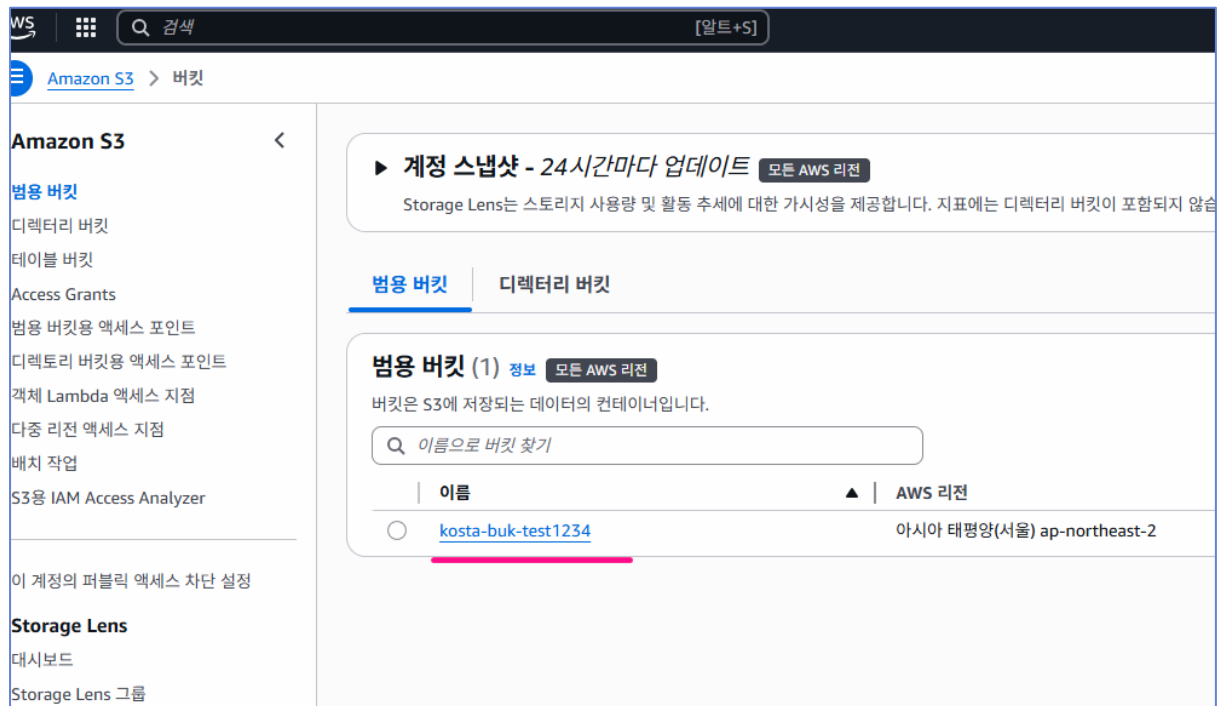
Bucket의 이름은 전 세계적으로 unique해야 하므로 중복되지 않은 이름을 생성한다.

```
# bucket 생성 명령어
aws s3 mb s3://kosta-buk-test1234

# 확인하기
aws s3 ls
```

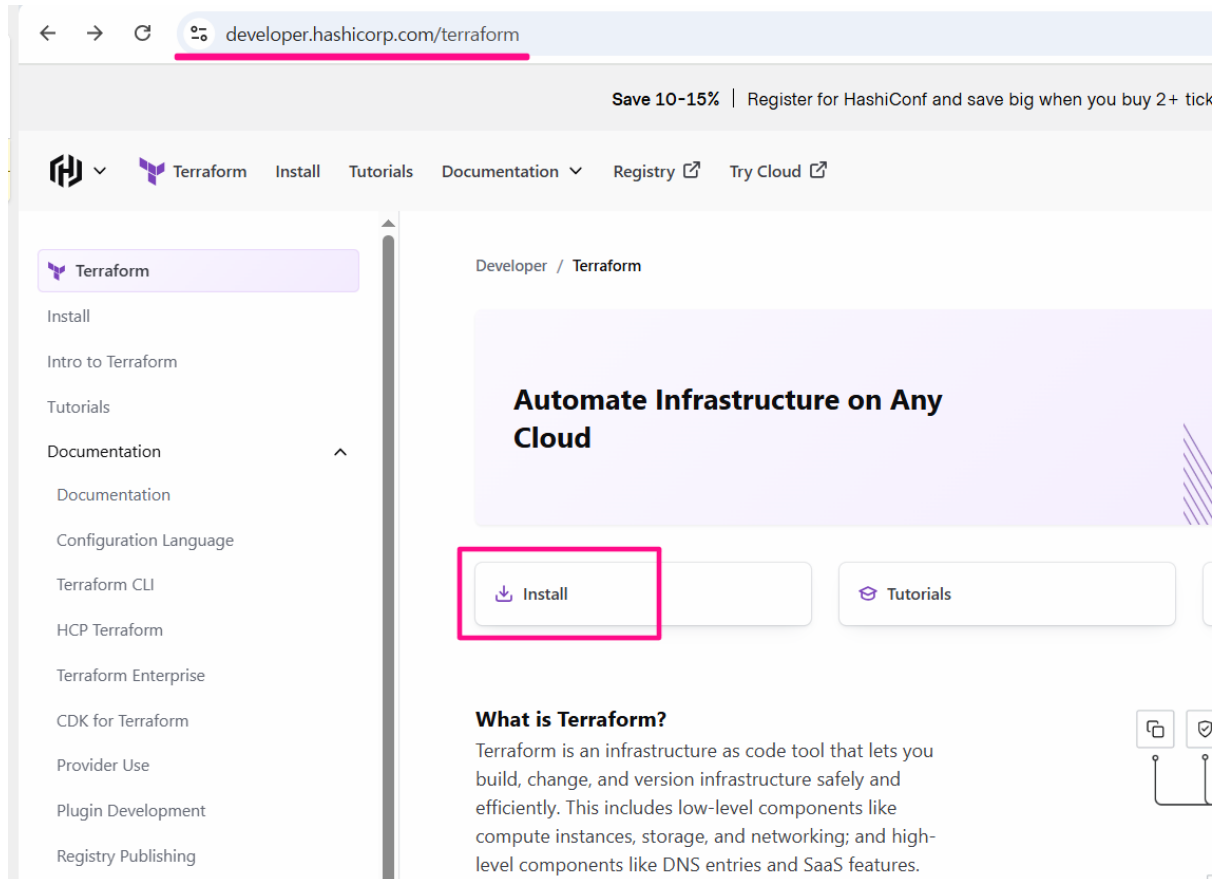
```
C:\Users\WKOSTA>aws s3 mb s3://kosta-buk-test1234
make_bucket: kosta-buk-test1234
```

AWS에서 확인해본다.



Terraform 설치하기

<https://www.terraform.io/downloads>



Save 10-15% | Register for HashiConf and save big when you buy 2+ tickets

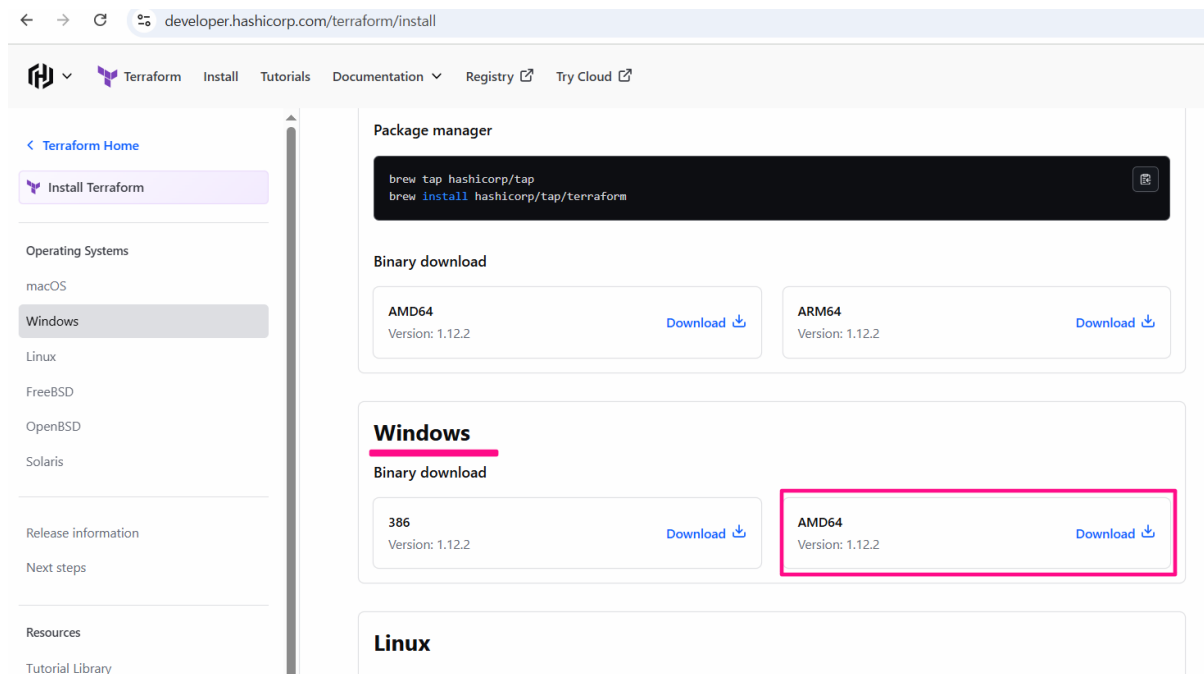
Developer / Terraform

Automate Infrastructure on Any Cloud

[Install](#) [Tutorials](#)

What is Terraform?

Terraform is an infrastructure as code tool that lets you build, change, and version infrastructure safely and efficiently. This includes low-level components like compute instances, storage, and networking; and high-level components like DNS entries and SaaS features.



developer.hashicorp.com/terraform/install

Package manager

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Binary download

Architecture	Version	Download
AMD64	Version: 1.12.2	Download
ARM64	Version: 1.12.2	Download

Windows

Binary download

Architecture	Version	Download
386	Version: 1.12.2	Download
AMD64	Version: 1.12.2	Download

Linux

다운로드 받는 파일을 압축 해제한다.

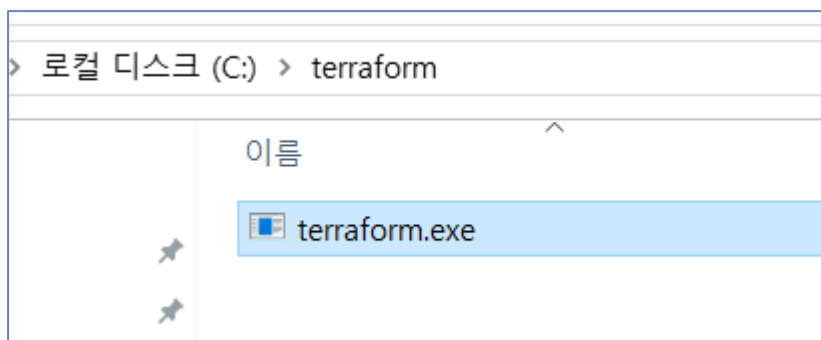
Windows 환경에서 terraform.exe는 직접 더블클릭해서 실행하는 실행파일이 아니라, 커맨드라인(cmd, PowerShell 등)에서 사용하기 위해 환경 변수(Path)에 등록해야 하는 도구이다.

적절한 위치로 이동

일반적으로 C:\terraform 또는 C:\Program Files\Terraform 같은 폴더를 만들어 그 안에 넣는 걸 권장

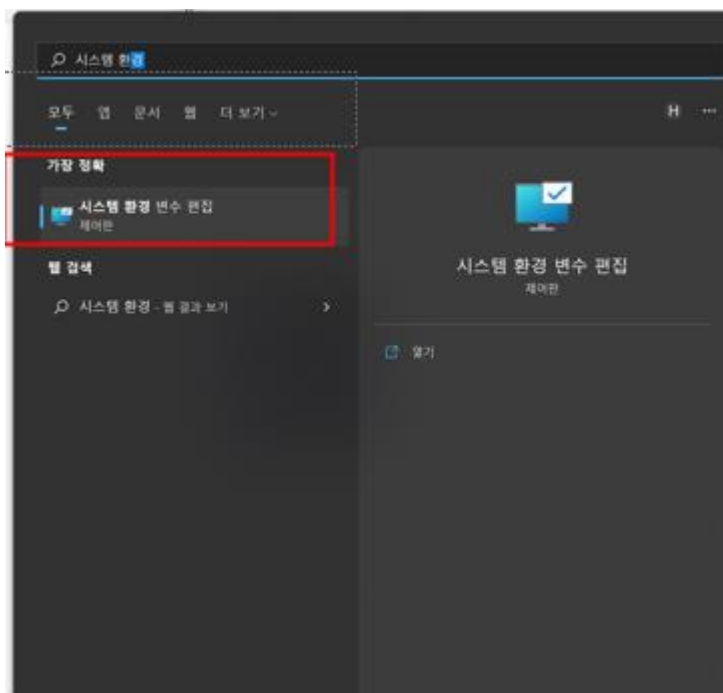
C:\terraform 경로를 만든다.

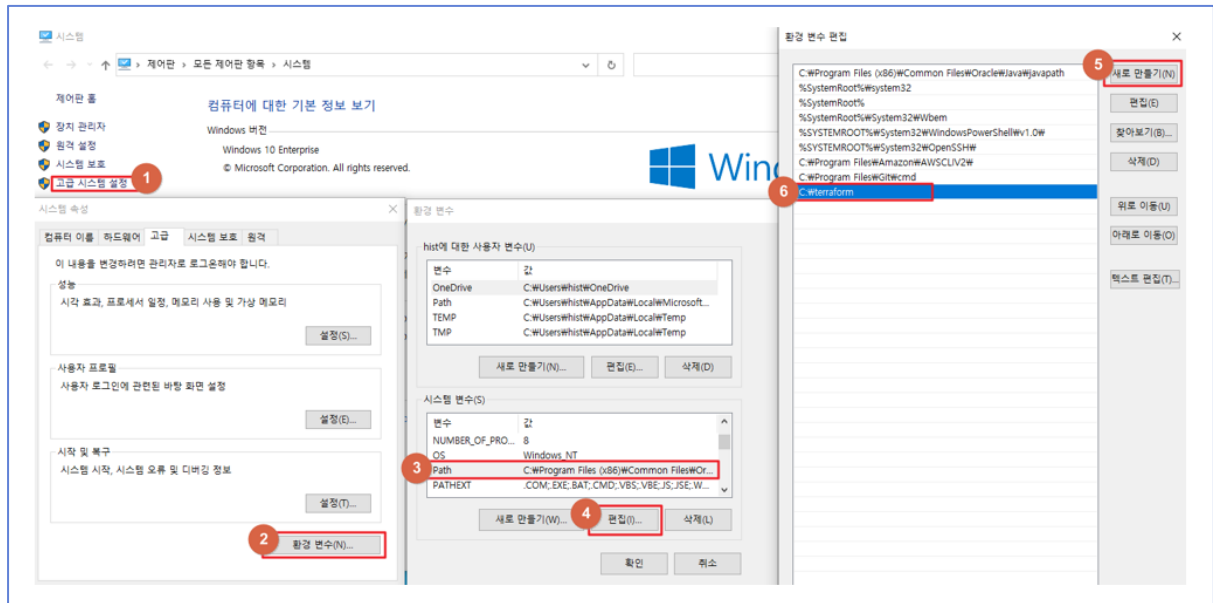
다운로드 받은 terraform.exe 파일을 c:\terraform경로에 복사 한다.



환경 변수 등록 (Path 설정)

시스템 환경변수 편집을 오픈



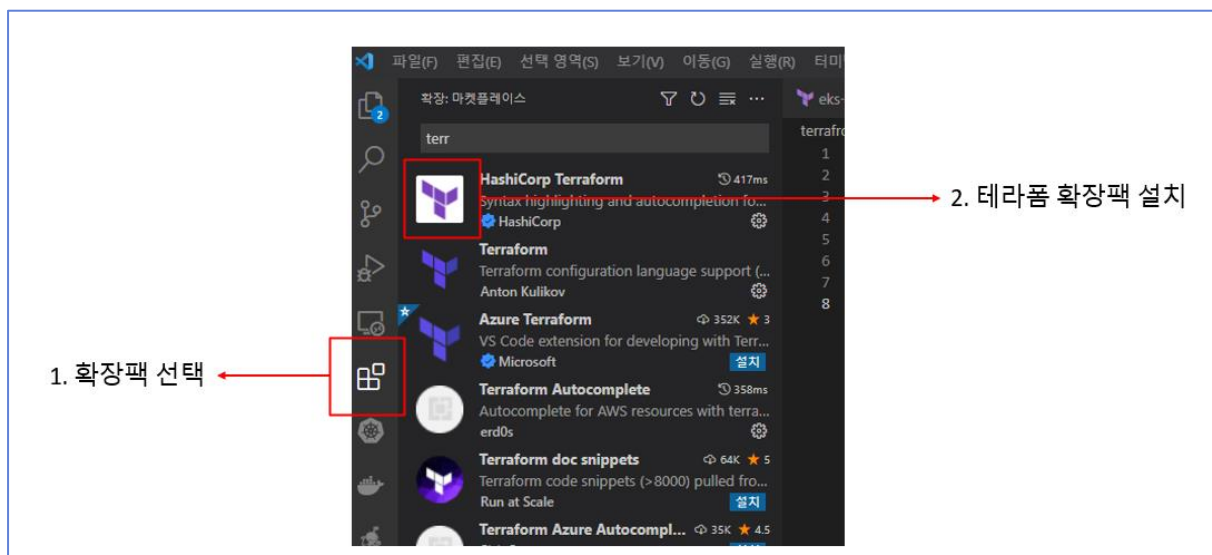


설정 후 새로운 cmd 열어서 확인하기

terraform -v

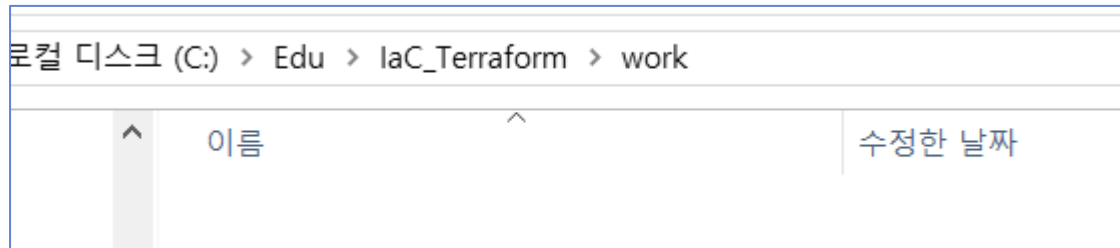
```
C:\Users\KOSTA>terraform -v
Terraform v1.12.2
on windows_amd64
```

VS Code 에 확장판 설치



terraform실습을 위한 작업 폴더를 만든다.

C:\Edu\WlaC_Terraform\work

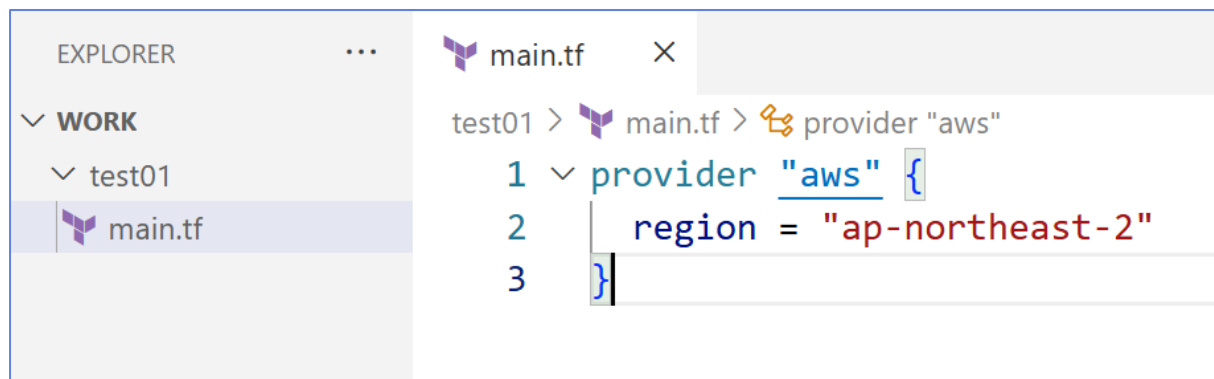


실습01

C:\Edu\IaC_Terraform\work 폴더를 vscode로 연다.

work폴더 > test01 폴더 생성

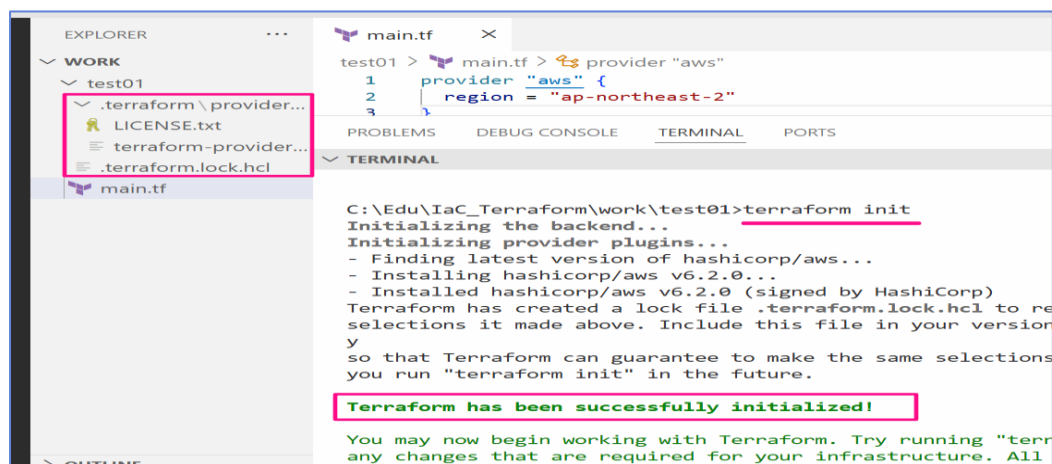
test01 > main.tf 파일 생성



provider는 테라폼으로 만들 Infrastructure의미한다 .

```
provider "aws" {
  region = "ap-northeast-2"
}
```

터미널을 열고(ctrl + `) > terraform init 실행



terraform init을 하면 폴더에 있는 *.tf 파일 내용을 참고해서 초기화 작업을 해준다.
이 명령어를 실행하기 위해서는 CLI 액세스 설정이 되어 있어야 한다.

S3 오프젝트 생성하기

work > test01 > s3.tf

```
resource "aws_s3_bucket" "s3 리소스 이름" {
  bucket = "버킷 이름"
}
```

리소스의 이름은 테라폼에서 관리하는 변수의 이름이고 bucket 뒤의 이름은 실제 버킷의 이름이다.
그래서 통상 변수의 이름과 버킷 이름을 같이 준다.

버킷이름은 unique해야한다.

```
test01 > s3.tf > resource "aws_s3_bucket" "kosta-295"
1 resource "aws_s3_bucket" "kosta-295" {
2   bucket = "kosta-295"
3 }
```

Test01터미널에서 terraform plan을 입력

오류 없이 Plan: 1 to add, 0 to change, 0 to destroy 메시지가 나오는지 확인

```
C:\Edu\IaC_Terraform\work\test01>terraform plan
```

```
Terraform used the selected providers to generate the following
execution plan. Resource actions are indicated with the following
symbols:
+ create
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Test01 터미널에서 terraform apply를 입력

Do you want to perform these action? 메시지에서는 yes를 입력

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

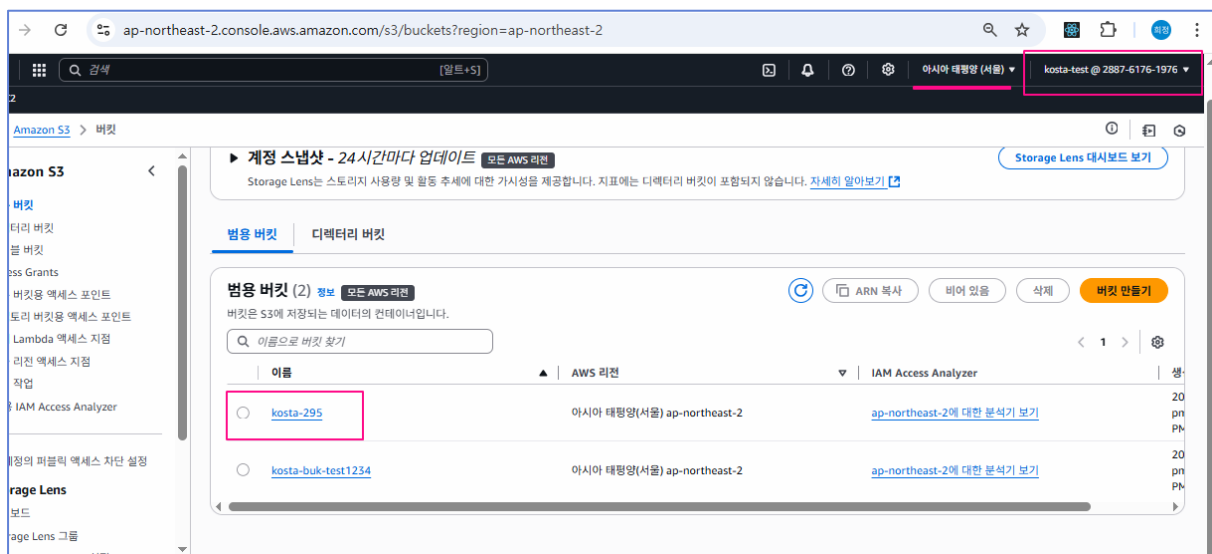
Enter a value: yes

aws_s3_bucket.kosta-295: Creating...

aws_s3_bucket.kosta-295: Creation complete after 1s [id=kosta-295]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

terraform plan > terraform apply > yes



CLI 명령어로 확인

aws s3 ls

```
C:\Edu\IaC_Terraform\work\test01>aws s3 ls
2025-07-08 15:16:51 kosta-295
2025-07-07 16:37:04 kosta-buk-test1234
```

Test01 터미널에서 terraform destroy 를 입력

Do you want to perform these action? 메시지에서 yes를 입력
kosta-295 s3 삭제됨

```
C:\Edu\IaC_Terraform\work\test01>terraform destroy
aws_s3_bucket.kosta-295: Refreshing state... [id=kosta-295]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# aws_s3_bucket.kosta-295 will be destroyed
- resource "aws_s3_bucket" "kosta-295" {
  - arn = "arn:aws:s3:::kosta-295" -> null
```

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```
aws_s3_bucket.kosta-295: Destroying... [id=kosta-295]
aws_s3_bucket.kosta-295: Destruction complete after 1s
```

Destroy complete! Resources: 1 destroyed.

코드로 만들어 놓으면 재사용 할 수 있다.

주의사항

테라폼으로 만든 것 들만 인지한다.

다시, terraform plan > terraform apply > yes 해보고 state list 확인해보자.

terraform state list

```
C:\Edu\IaC_Terraform\work\test01>terraform state list
aws_s3_bucket.kosta-295
```

실습2

Terraform을 이용하여

VPC , Subnet , InternetGateWay, Routing table, Security group , EC2 생성하기

제공파일 > test02 폴더를 work폴더로 복사한다.

terraform 초기화

terraform init

terraform으로 작성한 코드의 실행 예측결과 (*.tf)

terraform plan

terraform코드로 실제 인프라를 생성

terraform apply

terraform state 확인

terraform state list

생성된 자원들을 state파일 기준으로 모두 삭제

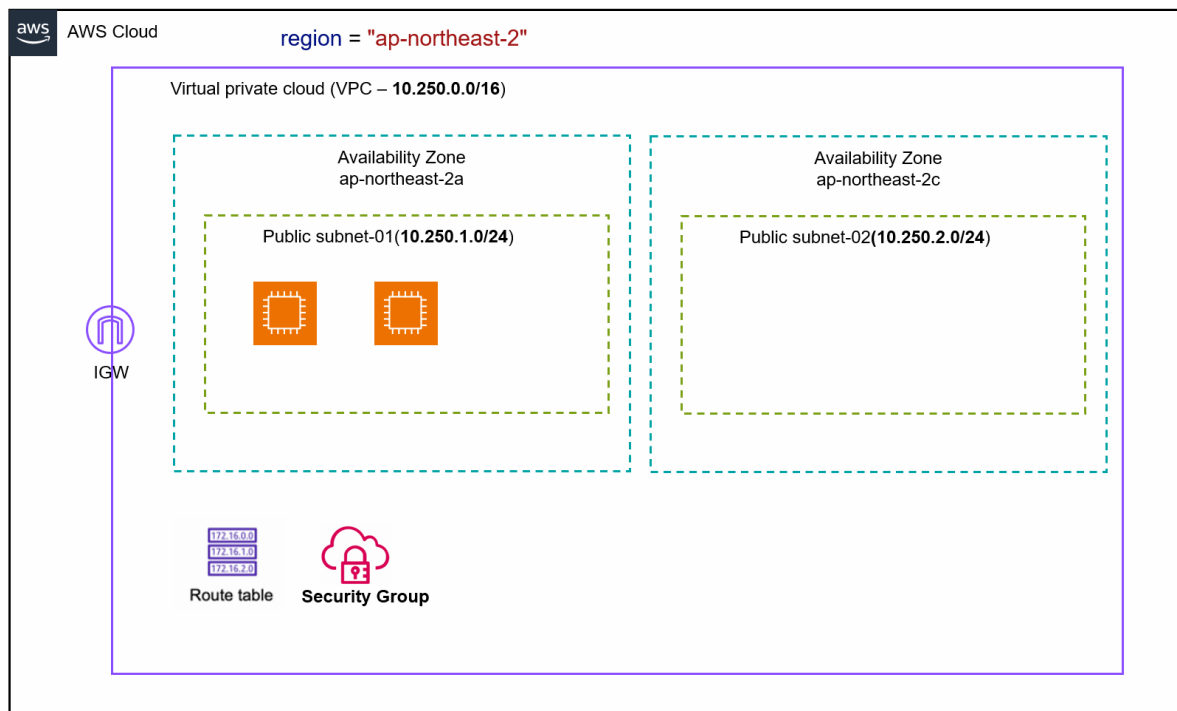
terraform destroy

```
C:\Edu\IaC_Terraform\work\test02>terraform state list
aws_instance.terraform-pub-ec2-bastion-2a
aws_internet_gateway.terraform-igw
aws_route.terraform-pub-rt
aws_route_table.terraform-pub-rt
aws_route_table_association.terraform-pub-rt-associate-2a
aws_security_group.terraform-sg-alb
aws_security_group.terraform-sg-bastion
aws_subnet.terraform-pub-subnet-2a
aws_vpc.terraform-vpc
```

test02 폴더 > Terraform 설정 정보

항목	이름	내용
01.Provider.tf		region = "ap-northeast-2"
02.vpc.tf	terraform-vpc	cidr_block = "10.250.0.0/16"
03.Subnet.tf	terraform-pub-subnet-2a	cidr_block = "10.250.1.0/24"
		availability_zone = "ap-northeast-2a"
	terraform-pub-subnet-2c	cidr_block = "10.250.2.0/24"
		availability_zone = "ap-northeast-2c"
04.igw.tf	terraform-igw	
05.Routing Table.tf	terraform-pub-rt	
06.Security Group.tf	terraform-sg-bastion	ingress
		from_port = 22
		from_port = 80
		from_port = 8080
		from_port = 443
		from_port = 3306
	terraform-sg-alb	egress
		from_port = 0 # 모든 영역
		ingress
		from_port = 80
07.ec2.tf	terraform-pub-ec2-front-2a	instance_type = "t3.small"
		subnet_id = aws_subnet.terraform-pub-subnet-2a.id
08.ec2.tf	terraform-pub-ec2-back-2a	instance_type = "t3.small"
		subnet_id = aws_subnet.terraform-pub-subnet-2a.id

test02 폴더 > Terraform 전체 구조



참고

<https://terraform101.inflern.devopsart.dev/>